

# Exercício Prático: API de Matrículas da Academia Lions

---

**Turma:** LionsDev

**Tópicos:** APIs REST com Express.js, Integração com MongoDB (Mongoose), Schemas e Modelos, Validações básicas com Mongoose, Operações CRUD assíncronas (async/await), Query Params (filtros), Regras de Negócio simples e Status Codes HTTP.

---

## 1. Contexto

A **Academia Lions** está recebendo novas matrículas e precisa organizar os dados dos alunos, planos contratados e valores a pagar. Hoje esse controle é feito manualmente, o que dificulta saber quem está ativo, pausado ou cancelado.

Você deverá criar uma API REST simples conectada ao MongoDB para gerenciar matrículas da academia. A estrutura deve seguir o modelo usado em aula: `server.js`, `db.js` e `models/`, com as rotas escritas diretamente no `server.js`.

---

## 2. Configuração Inicial

Crie uma API Node.js com Express e Mongoose.

Requisitos estruturais:

- Crie um arquivo `.env` contendo as variáveis `MONGO_URI` e `PORT` (porta `3000`).
- Crie a pasta `src`.
- Crie o arquivo `src/db.js` para gerenciar a conexão com o MongoDB.
- Crie o arquivo `src/server.js` como ponto de entrada da aplicação.
- Crie a pasta `src/models` e nela o arquivo `matricula.js`.
- Todas as rotas devem ficar no `src/server.js`, como fizemos em sala.

### O Modelo (Schema) da Matrícula

O Schema do Mongoose para as `matriculas` deve conter os seguintes campos:

- **nomeAluno** : Tipo **String** , obrigatório.
- **idade** : Tipo **Number** , obrigatório.
- **modalidade** : Tipo **String** , obrigatório (deve aceitar apenas: **Musculação** , **Funcional** ou **Dança** ).
- **plano** : Tipo **String** , obrigatório (deve aceitar apenas: **Mensal** , **Trimestral** ou **Semestral** ).
- **dataMatricula** : Tipo **String** ou **Date** , obrigatório (ex: **"2026-06-15"** ).
- **valorMensal** : Tipo **Number** (será calculado automaticamente pela API).
- **valorTotal** : Tipo **Number** (será calculado automaticamente pela API).
- **status** : Tipo **String** , com valor padrão de **"Ativa"** (deve aceitar apenas: **Ativa** , **Pausada** ou **Cancelada** ).

## 3. Requisitos Obrigatórios e Regras de Negócio

### 3.1 Cadastrar Matrícula (CREATE)

Crie a rota **POST /matriculas** .

O corpo da requisição enviará os dados da matrícula (exceto **valorMensal** , **valorTotal** e **status** ).

```
{
  "nomeAluno": "Beatriz Lima",
  "idade": 17,
  "modalidade": "Funcional",
  "plano": "Trimestral",
  "dataMatricula": "2026-06-15"
}
```

Regras:

1. **Definição do Valor Mensal:** Antes de salvar no banco, o backend deve preencher **valorMensal** seguindo esta tabela:
  - **Musculação** : R\$ 90
  - **Funcional** : R\$ 120
  - **Dança** : R\$ 100
2. **Cálculo do Valor Total:**
  - Plano **Mensal** : 1 mensalidade.

- Plano **Trimestral** : 3 mensalidades com 10% de desconto.
- Plano **Semestral** : 6 mensalidades com 15% de desconto.

3. **Retorno**: Salve a matrícula no banco e retorne o documento criado com status **201** .

### 3.2 Listar Todas as Matrículas (READ)

Crie a rota **GET /matriculas** .

Regras:

- A rota deve retornar todas as matrículas salvas no MongoDB usando **Matricula.find()** .
- Retorne o array de resultados com status **200** .

### 3.3 Buscar Matrículas por Modalidade (QUERY PARAMS)

Crie a rota **GET /matriculas/busca** .

Esta rota deve aceitar um filtro opcional via Query Params chamado **modalidade** :

- Exemplo de URL: **http://localhost:3000/matriculas/busca?modalidade=func**
- A busca deve retornar todas as matrículas em que a modalidade contenha o texto pesquisado.

### 3.4 Atualizar Status da Matrícula (UPDATE)

Crie a rota **PATCH /matriculas/:id** .

O corpo da requisição deve enviar apenas o novo status (ex: **{ "status": "Pausada" }** ).

Regras:

- Busque a matrícula pelo ID e atualize o status usando **findByIdAndUpdate** .
- Se o ID não for encontrado, responda com status **404** .
- Retorne o documento atualizado com status **200** .

### 3.5 Remover Matrícula (DELETE)

Crie a rota **DELETE /matriculas/:id** .

Regras:

- Remova a matrícula do banco de dados pelo ID usando **findByIdAndDelete** .

- Se não encontrar o registro, retorne status **404**.
  - Se remover com sucesso, retorne status **200** com uma mensagem de confirmação.
- 

## 4. Testes Esperados

Realize testes na sua API seguindo este fluxo de validação:

1. Cadastre uma matrícula **Mensal** de **Musculação** e verifique se o total ficou R\$ 90.
  2. Cadastre uma matrícula **Trimestral** de **Funcional** e verifique se o desconto de 10% foi aplicado.
  3. Liste todas as matrículas cadastradas.
  4. Faça uma busca pela modalidade **func**.
  5. Atualize o status de uma matrícula para **Pausada**.
  6. Delete uma das matrículas informando o ID.
- 

## 5. Dicas para a Implementação

- Use **if** ou **switch** para calcular o valor mensal da modalidade.
  - Para o plano trimestral, calcule **valorMensal \* 3** e aplique desconto.
  - Para o plano semestral, calcule **valorMensal \* 6** e aplique desconto.
  - No **findByIdAndUpdate**, passe **{ new: true, runValidators: true }**.
  - Lembre-se de usar **app.use(express.json())** no **src/server.js** para conseguir ler o corpo das requisições.
- 

**LionsDev** • Professor Nicolas Cardoso Motta  
*Exercício Prático de Mongoose e MongoDB - Módulo 08*